

NCHU - [DRL][FR] Final Project Phase 2-change02

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明，也把三個附檔都過了一遍。我的判斷很明確：

最適合你拿來做這門 DRL final project proposal 的題目，是：

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制（Adaptive Traffic Signal Control）」。

你附的兩份 drone reference，其實是在示範同一種教授喜歡的敘事方式：先把問題的重要性講大，再整理 related work，接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**，還把資料量、模型、限制都量化寫出來；Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文，但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本，這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**；搭配你上傳的 mp4 內容，我理解教授現在要的不是「先把系統全部做完」，而是要你先把整條論文故事線講順：**這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚，就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**；而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它**最貼近教授 reference 的味道**。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它**非常適合 A-to-Z 的 proposal 寫法**。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。📄 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方面，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based
- RL / DRL traffic signal control：Q-learning、DQN、PPO、multi-agent
- reward / state design：queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment：SUMO / SUMO-RL
- Agent：先用 DQN 或 PPO
- State：各車道 queue length、waiting time、current phase、phase elapsed time
- Action：切換到哪個 phase，或維持 / 切換
- Reward：負的總等待時間 + queue penalty
- Baselines：fixed-time、actuated control
- Metrics：average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：
題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「**這個題目能不能當 final project proposal**」：可以。

如果你是問「**可不可以直接當 DRL 專案原封不動拿去做**」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。 [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。 [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的設計痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。 [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」 [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你把这个題目改寫成一份**符合教授 A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design**。

另外如果是 MemPalace 這個 AI Agent 永久記憶開源專案呢 是否適合當作題目
且因為 MemPalace 的亮點就是能把 AI 的記憶 "本地化" 如果這個本地化模型還要跑重武器 DRL 有點本末倒置 (本地化就是要越輕量越好)
所以我想從「輕量化 DRL」的切入方向
不要用深度神經網絡 (Deep Q-Network)，改用 Multi-Armed Bandit (MAB, 多臂老虎機) 或 輕量級 Q-Learning

但我連 ai agent 的架設都不會 也沒用過 這樣的程度適合做這個題目嗎
不太適合的話是否有折衷方案 連 ai agent 概念都不用摸呢
[顯示更多顯示較少](#)

可以，但我會先給你一個很務實的判斷：

MemPalace 適合當靈感來源，也適合當研究對象；但不適合你現在這個程度直接做成「完整 AI agent 記憶系統實作」的 final project。

因為 MemPalace 本身不是小玩具，它是完整的 **local-first AI memory** 系統：用語意搜尋保留逐字記憶、預設後端是 ChromaDB、知識圖譜用本機 SQLite，還有 MCP 工具、hooks、agent diary、benchmark 等完整配套。官方 repo 也主打本地化、零 API、LongMemEval raw R@5 96.6%。[GitHub+2](#)

你抓的方向其實是對的：**如果題目核心價值是「本地化、輕量化、低依賴」**，那再往上疊一個需要重訓練、重推論的 DQN，確實很可能會讓題目失焦。MemPalace 官方自己也一直強調 bounded wake-up context，大約只載入 L0+L1 的 ~600-900 tokens，且搜尋是按需觸發，不是整包上下文全塞進去；這種設計哲學本來就偏向「資源節制」而不是「深模型硬壓」。這是你可以在 proposal 裡合理主張的切入點。[mempalaceofficial.com+2](#)

但要注意一個關鍵：**MAB 或 tabular Q-learning 並不等於 Deep RL。**

Contextual bandit 比較像是「每回合看到 context，選一個 action，只看到被選 action 的 reward」，是介於 supervised learning 與 RL 之間的問題；tabular Q-learning 則是完整 RL，但它假設你有可管理的有限 state/action，透過探索與更新表格來學策略。官方與教材都把 contextual bandit 和 Q-learning 定義得很清楚，所以如果你的課名是 *Deep Reinforcement Learning*，你若只做純 MAB，教授有機會質疑「這題不夠 deep」。[Microsoft GitHub+2](#)

所以我對你這題的建議不是「放棄 MemPalace」，而是：

最適合你的版本

不要做完整 agent。改做：

「本地記憶系統中的輕量化決策層」

也就是把 MemPalace 當成既有記憶後端，你的研究重點不是重造一個 agent，而是研究：

在 token budget、延遲限制、不同 query 類型下，
要用哪一種記憶存取策略最划算？

這樣你的題目就會很乾淨。

我最推薦的題目版本

題目 A：最穩、最適合你

A Contextual Bandit for Token-Budgeted Local Memory Retrieval

白話版就是：

每來一個 query，bandit 根據 context 決定要走哪條 retrieval 路線。

例如 action 可以是：

- 只用 wake-up
- 只用 search
- 先 wake-up 再 search
- 只查某個 wing
- 查 knowledge graph
- top-k 取 3 / 5 / 10

這些 action 都很小、很離散，很符合 contextual bandit「選有限個 action、看 chosen action reward」的型態。MemPalace 官方也提供了不用 agent 的 CLI 與 Python API：你可以直接跑 `mempalace wake-up`、`mempalace search`，或用 `search_memories()` 之類的 Python API 把結果接進你自己的實驗程式，不一定要碰 MCP agent。🔗 mempalaceofficial.com+2

這題最大的優點是：

你完全可以不會架 AI agent，也能做。

第二順位：稍微更像 RL

題目 B：進階但還可控

Tabular Q-Learning for Multi-Step Local Memory Access Policies

這版把問題變成多步決策，例如：

1. 先讀 wake-up

2. 判斷要不要搜 wing
3. 搜不到再深搜
4. 最後決定回傳多少內容

這時候就不是 one-shot bandit，而是有狀態轉移的 sequential decision problem，tabular Q-learning 在概念上比較合理。Q-learning 本來就是不需要先知道完整 MDP，就能從互動資料學 action-value；而且 Gymnasium 也有官方 tabular Q-learning 教學與自訂環境入口，對學生來說比 DQN 友善很多。

 Dive into Deep

Learning+1

但這題比題目 A 稍難，因為你要自己定義 state transition。

你現在的程度，適不適合？

我的判斷是：

做「完整 MemPalace agent + 本地模型 + MCP + hooks + RL」：不適合。

MemPalace 官方文件已經很明白，它平常的使用方式會牽涉到 MCP integration、Claude Code plugin、Gemini CLI、hooks；而「specialist agents」目前官方 docs 主要描述的是透過 MCP tools 寫 diary 的 workflow，不是那種你開箱即用就有完整 agent orchestration 的教學專案。對一個還沒碰過 AI agent 架設的人，這個入口太陡。  mempalaceofficial.com+2

做「MemPalace 當後端 + 你自己寫一個輕量 decision layer」：適合。

因為你可以把所有 agent 概念先拿掉，只保留：

- query
- retrieval action
- reward
- evaluation

這樣本質上就是一個 machine learning / RL 實驗題，不是 agent engineering 題。

如果你想連 agent 概念都不要碰

那我建議你直接再退一步，做成：

題目 C：最保守、最容易成功

Lightweight RL for Local Memory Retrieval Strategy Selection

這題甚至可以不真的安裝 MemPalace。

你只要自己準備一個小型資料集，例如：

- 一批對話紀錄 / 筆記 / 文件片段
- 一組 query

- 每個 query 對應的 relevant memory 片段標註

然後模擬幾種 retrieval strategy：

- 全域搜尋
- 分 room 搜尋
- 限 top-k
- 先短上下文再深搜
- 查知識圖譜/metadata 或不查

最後用 contextual bandit 或 tabular Q-learning 去學「哪種 query 應該走哪種策略」，reward 可以是：

$$\text{reward} = \text{recall@k} - \alpha \cdot \text{token_cost} - \beta \cdot \text{latency}$$

這樣你：

- 不用會 agent
- 不用會 MCP
- 不用接本地 LLM
- 甚至不用碰 MemPalace 真正的工程堆疊

但論文故事仍然成立，而且非常貼近你在意的「本地化」「輕量化」「不要重武器」。這個方向也跟 contextual bandit 在互動式推薦、客服 agent 裡的實際應用邏輯很接近；Microsoft 就有把 contextual bandits / MAB 用在 virtual agent 的推薦與意圖消歧上，重點正是 partial feedback 與資料效率。 [arXiv+1](#)

我給你的最終建議

若以你現在的程度，我會這樣排序：

最推薦：

題目 A：Contextual Bandit for Token-Budgeted Local Memory Retrieval

因為最輕、最不需要 agent、最符合你要的本地化邏輯。

次推薦：

題目 C：完全不碰 agent 的 retrieval strategy selection

最保守，成功率最高。

不推薦現在做：

完整 MemPalace agent 系統 + RL 強化

因為你會先死在系統整合，不是死在研究本身。

一句話結論

MemPalace 可以做，但不要把題目定成「做一個 AI agent 永久記憶系統」。

對你最好的版本是把它改成：

「本地記憶系統中，如何用輕量化 RL / contextual bandit，在 token 與延遲限制下選擇最佳 retrieval 策略？」

這樣你幾乎可以完全不碰 **agent 概念**，卻還是保留 MemPalace 的亮點與研究價值。官方 GitHub 就是 [MemPalace/mempalace](#)。 

下一步最有幫助的做法，是我直接幫你把這題寫成教授 A-to-Z 架構的 **final project proposal 題目 + abstract + 方法設計**。

貼上的 Markdown (1).md

檔案

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

檔案

Smart_City_Drone_Law_Enforcement.pdf

PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

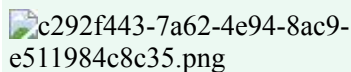
1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 ****"openspec"**) 給我**

3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色
顯示更多顯示較少

16\ 給 AntigraVity 的 change-02 指令草案

c292f443-7a62-4e94-8ac9-e511984c8c35.png

附圖以及附檔 是我當初下 Prompt 讓你完成任務的螢幕截圖的時空背景 還有檔案 我現在再補上一次 你可以把它當作溫故知新

然後直接執行以下

用繁體中文給我這個「**16. 給 AntigraVity 的 change-02 指令草案**」 我要可以讓 AntigraVity 最精準執行任務的「最完美且超高品質的**最佳 Prompt**」

備註：

因為我之前發生過我跟一位 AI Agent 協作

我也是請他用官方 OpenSpec 套件用 SDD 開發模式幫我實作專案

結果他竟然用的不是官方 OpenSpec

我問他為什麼並且一直以來他用的是什麼

他竟然跟我說他是去模擬 OpenSpec 官方形容的 SDD 開發模式

他自己再用一樣的方式刻出來

等於只是複製了 OpenSpec SDD 其中的 workflow 而已

根本沒有真正用到 OpenSpec 套件工具

他跟我說這叫什麼棕色模擬 我不管他叫什麼色

他就是學了皮膚再自己的方式去套用

這方法一點都沒有經過實戰驗證 至少 OpenSpec 是有大量使用社群

自己亂規範自己 寫出來的程式一點可信度都沒有

我要叫 Antigravity 幫我安裝

我跟 Antigravity 協作的方式要以官方 OpenSpec 套件當作 SDD 的實現工具

OpenSpec 也是我跟 Antigravity 對齊專案方向的重要規格

你生給我的最佳 Prompt 要包含 OpenSpec 官方的指令 cli api 等等請 Antigravity 實作

因為是 Antigravity **開工前最佳 Prompt** 要有一槌定音的效果

請用最高規格對待 思考縝密 完備 完美執行生成**最佳 Prompt** 的任務

我要用的是「**官方 OpenSpec 套件**」

顯示更多顯示較少

16\ 給 Antigravity 的 change-02 指令草案

 f0ddb2ce-26a1-4ef2-969b-759e7e65bfb3.png

 **From 0 to 1 Deep RL with SMC MDP.pdf**
PDF

 **media.mp4**
檔案

 **2330_SMC_FVG_MTF_Report_A4_Portrait.pdf**
PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

附圖以及附檔 是我當初下 Prompt 讓你完成任務的螢幕截圖的時空背景 還有每份檔案 我現在再補上一次 你可以把它當作對這份期末專題的溫故知新

然後直接執行以下

用繁體中文給我「**16. 給 Antigravity 的 change-02 指令草案**」的**正式最佳 Prompt** 不僅僅只是草案 我要可以讓 Antigravity 最精準執行任務的「最完美且超高品質的**最佳 Prompt**」

備註：

因為我之前發生過我跟一位 AI Agent 協作

我也是請他用官方 OpenSpec 套件用 SDD 開發模式幫我實作專案

結果他竟然用的不是官方 OpenSpec

我問他為什麼並且一直以來他用的是什麼

他竟然跟我說他是去模擬 OpenSpec 官方形容的 SDD 開發模式

他自己再用一樣的方式刻出來

等於只是複製了 OpenSpec SDD 其中的 workflow 而已

根本沒有真正用到 OpenSpec 套件工具

他跟我說這叫什麼棕色模擬 我不管他叫什麼色

他就是學了皮膚再自己的方式去套用

這方法一點都沒有經過實戰驗證 至少 OpenSpec 是有大量使用社群

自己亂規範自己 寫出來的程式一點可信度都沒有

我要叫 Antigravity 幫我安裝

我跟 Antigravity 協作的方式要以官方 OpenSpec 套件當作 SDD 的實現工具

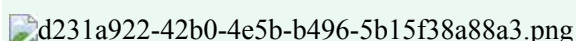
OpenSpec 也是我跟 Antigravity 對齊專案方向的重要規格

你生給我的最佳 Prompt 要包含 OpenSpec 官方的指令 cli api 等等請 Antigravity 實作

因為是 Antigravity ****開工前最佳 Prompt**** 要有一槌定音的效果

請用最高規格對待 思考縝密 完備 完美執行生成****最佳 Prompt**** 的任務

我要用的是「****官方 OpenSpec 套件****」
顯示更多顯示較少



我已確認 Change 02 的問題與 Change 01 revisions 狀態。

結論：

Change 01 revisions 通過，project-charter 可視為正式完成。

專案可以推進到 Change 02：ns3-baseline-benchmark。

但 Change 02 在進入 implementation / /opsx:apply 前，請先依照以下決策修正 proposal / design / specs。

1. proposal.md：Why / Capabilities / Impact / Acceptance Criteria

可接受。

proposal.md 的方向正確：

- Change 02 聚焦 ns-3 baseline benchmark
- 不碰 RL
- 不碰 ns3-gym
- 不碰 DQN / PPO
- 不做 DRL vs baseline
- NewReno + CUBIC 為 required baseline
- BBR 為 strongly preferred but non-blocking
- 需要輸出 CSV log、baseline figures、summary table

但 Acceptance Criteria 請修正兩點：

A. Reproducibility 不要要求 FlowMonitor XML byte-for-byte identical。

請改成：

The same scenario with the same random seed SHOULD produce metric-equivalent outputs within documented tolerance.

B. Utility score 權重只能是 preliminary / provisional。

請補註：

Change 02 的 utility_score 只作為 baseline visualization metric。公式與權重是 provisional，最終 reward / utility 權重可在 Change 04 / Change 05 經 Spec Owner approval 後調整。

2. design.md：D-04 BBR decision gate / D-01 router-based topology

可接受。

D-01 router-based topology 接受：

- 允許合理的 ns-3 router-based single bottleneck topology
- router-based topology 不違反 single bottleneck path
- 禁止的是 multi-bottleneck、multi-path、multiple sender/receiver groups、large-scale topology

D-04 BBR decision gate 接受：

- NewReno + CUBIC 是 required
- BBR 是 strongly preferred but non-blocking
- 若 BBR 整合成本過高或版本不可用，不得阻塞 NewReno / CUBIC baseline
- BBR 可移至 optional / future work，但必須文件記錄原因

3. spec.md：SHALL / MUST 要求

大致可接受，但請修正兩條：

A. 不要寫：

byte-for-byte equivalent FlowMonitor XML

請改成：

metric-equivalent outputs under the same random seed, within documented tolerance

B. Utility score 不要寫成 final requirement。

請改成：

utility_score SHALL be included as a preliminary baseline visualization metric. The formula and weights are provisional and may be revised in Change 04 / Change 05 with Spec Owner approval.

其餘 SHALL / MUST 方向可接受：

- MUST create configurable single bottleneck topology
- MUST run NewReno and CUBIC
- BBR SHALL run if available, otherwise documented as non-blocking optional
- MUST output throughput / RTT / loss CSV
- MUST generate baseline figures
- MUST NOT include RL / ns3-gym / DQN code in Change 02

4. OQ-02.01：ns-3 目標版本

正式決策：

使用 ns-3.40 作為 Change 02 / Change 03 的目標版本。

理由：

- 後續 Change 03 需要銜接 ns3-gym
- ns3-gym 官方 README 直接示範使用 ns-allinone-3.40，並搭配 app-ns-3.36+ branch
- 因此 ns-3.40 比最新穩定版更適合本專案的低風險路徑

請不要自行改用 ns-3.35、3.36 或最新穩定版。

若 ns-3.40 在實際環境無法穩定執行，請停止並回報 Spec Owner，不得自行升降版本。

5. Node.js 升級

正式決策：

Change 02 proposal 可以繼續；但在 Change 02 implementation / /opsx:apply 前，必須升級 Node.js 至 v20.19.0+。

本次不授予 waiver。

請在 implementation 前回報：

- node -v
- openspec --version
- 是否仍有 EBADENGINE warning

6. 是否可進入 Change 02

可以推進到 Change 02，但目前只批准 Change 02 proposal revisions。

請先完成以上修正，再回報：

“Change 02 proposal revisions complete, ready for Spec Owner review before /opsx:apply.”

注意：

在我驗收 Change 02 proposal revisions 前，不得開始 ns-3 coding，不得執行 benchmark，不得建立 experiment results，不得寫 analysis script，不得進入 ns3-gym / DQN。

以上是我已經請 Antigravity 做完 Phase 2-change01 後

Antigravity 要我回覆的五個問題 這五題我都是請 Chatgpt 幫我回答

而分割線以上就是我請 Chatgpt 產生出來的 Phase 2-change01 回饋並準備進入 Phase 2-change02

你幫我判斷我應該

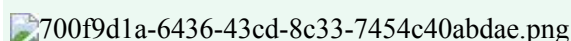
「包括你的 change-02 正式最佳 Prompt + Chatgpt Phase 2-change01 回饋」**一起餵給** Antigravity

還是

「你的 change-02 正式最佳 Prompt」以及「Chatgpt Phase 2-change01 回饋」**分開餵給** Antigravity

你覺得哪個方式比較好

顯示更多顯示較少

700f9d1a-6436-43cd-8c33-7454c40abdae.png

<https://github.com/donuop35/ns3-drl-congestion-control/commit/03b6f0daa397b87bf19a88d832a31a84beebd9c>

github 已推上 Phase Change 02 的異動 請幫我驗收是否全部 Change 02 要求都有通過

並且附圖是 Antigravity 要問我的問題 請幫我回覆

可以，最小幅度調整就是：把原本「Node.js 必須升級，但先回報」改成「現在授權你直接升級 Node.js 到 v20.19.0+，但升級後仍只能做 pre-apply cleanup，不准 /opsx:apply。」

OpenSpec 官方 Quick Start 明確要求 **Node.js 20.19.0 or higher**，並以 `npm install -g @fission-ai/openspec@latest`、`openspec init` 作為官方安裝與初始化流程；所以直接授權 Antigravity 升級 Node.js 是合理的，不需要 waiver。  [GitHub](#)

你只要把原本 Prompt 的 **第 3 段 Node.js upgrade decision** 和後面的 **Pending pre-apply cleanup** 小幅替換成下面這版即可。

3. Node.js upgrade decision

正式確認：

目前 Change 02 proposal revisions 可以通過。

現在授權你直接將 Node.js 升級至 v20.19.0+，以符合官方 OpenSpec requirement。

本次不授予 waiver，也不需要再詢問是否升級；請直接完成升級。

升級原則：

- 優先使用目前環境中最穩定、最不破壞專案的 Node.js 版本管理方式。
- 若環境已使用 nvm / fnm / asdf，請優先使用既有版本管理工具升級。
- 若環境不支援版本管理工具，請先回報目前環境限制，不得用高風險方式破壞系統 Node.js。
- Node.js 版本必須達到 v20.19.0+。
- 升級後請重新確認 OpenSpec CLI 可正常運作。
- 若升級後仍出現 EBADENGINE warning，必須停止並回報。

升級後請回報：

- node -v
- npm -v
- openspec --version
- 是否仍有 EBADENGINE warning
- 是否可執行 openspec validate / openspec status

注意：

Node.js 升級是 pre-apply cleanup 的一部分，不代表批准進入 implementation。
即使 Node.js 升級成功，在 Spec Owner 再次驗收前，仍不得執行 /opsx:apply。

然後把原本的 Pending pre-apply cleanup 改成這版：

目前狀態為：

Change 02 proposal revisions accepted.

Pending pre-apply cleanup:

- Directly upgrade Node.js to v20.19.0+
- Confirm OpenSpec CLI still works after Node.js upgrade
- Update tasks.md for 5 new spec validation tasks
- Run OpenSpec validation after tasks.md update

完成以上事項後，請回報：

Change 02 pre-apply cleanup complete, ready for Spec Owner approval before /opsx:apply.

並附上：

1. node -v
2. npm -v
3. openspec --version
4. 是否仍有 EBADENGINE warning
5. tasks.md 更新摘要
6. openspec validation warning / error 摘要

在我再次驗收前，不得執行 /opsx:apply，不得開始 ns-3 coding，不得執行 benchmark，不得建立 experiment results，不得寫 analysis script，不得進入 ns3-gym / DQN / PP0。

最終判斷：不需要整份 Prompt 重寫。

只要把「請在 implementation 前回報」改成「現在授權直接升級，升級後回報」，並保留「不得 /opsx:apply」這條線即可。

<https://github.com/donuop35/ns3-drl-congestion-control/commit/d8f093264bb51ca995df4635bd4fb02d9617e3b9>

Antigravity 又做完上個指令了並已推上 github Phase 2 change-02 的異動

請幫我驗收 git diff 是否全部 Change 02 要求都有通過

是否能推進到 Phase 2 change-03 了呢
顯示更多顯示較少

結論：這次 commit d8f0932... 的方向是正確的，但我不建議現在就直接推進到 Phase 2 change-03。

原因不是它做錯，而是 **Change 02 pre-apply cleanup** 還差一個硬性驗收點沒有完成：**0.10 openspec status ... 仍是未勾選**。目前 commit 已經補上 Node.js 升級、5 個 spec validation tasks、ns-3.40 freeze、reproducibility tolerance、utility provisional 等關鍵修正，但 **tasks.md** 自己仍標示「必須在 **/opsx:apply** 前完成」的 **0.10** 尚未完成，**0.11 Spec Owner** 再次驗收 也尚未完成。

[pre-apply cleanup: Node.js upgr...](#)

補充：你先前上傳的一些附件目前系統顯示已過期；但本次驗收主要依 GitHub commit diff 與官方文件即可完成。

Change 02 本次 commit 驗收結果

項目	驗收結果	判斷
Node.js v20.19.0+	通過但需看實際回報	commit message 寫已升級到 v20.20.2, tasks.md 也勾選 Node.js 已升級、 openspec --version 可執行、無 EBADENGINE warning。 pre-apply cleanup: Node.js upgr...
5 個 spec validation tasks	通過	已新增並勾選 baseline-methods、topology、metrics-logging、scenario-matrix、benchmark-risk-register 五項 validation tasks。 pre-apply cleanup: Node.js upgr...
ns-3.40 freeze	通過	已把原本「ns-3 >= 3.32」修正為明確要求 ns-3.40 ，並寫明不能用 3.35、3.36 或 latest stable。 pre-apply cleanup: Node.js upgr...
BBR availability check	通過	已改為在 ns-3.40 中確認 TcpBbr class 是否存在於 src/internet/model/tcp-bbr.h 。 pre-apply cleanup: Node.js upgr...
reproducibility 修正	通過	已從「identical FlowMonitor output」改為同 seed 下 metric-equivalent outputs，且明確寫 byte-for-byte XML identity 不需要。 pre-apply cleanup: Node.js upgr...
utility_score provisional	通過	已補上 utility formula 是 provisional, weights 不是 final, Change 04/05 可經 Spec Owner approval 調整。 pre-apply cleanup: Node.js upgr...
OpenSpec status / validation 回報	未通過 / 缺證據	0.10 Run openspec status ... 仍未勾選，代表它自己也還沒有完成 pre-apply cleanup 的最後檢查。 pre-apply cleanup: Node.js upgr...
是否可 /opsx:apply	不可	0.11 明確寫 Spec Owner 再次驗收通過才可 /opsx:apply ，目前尚未勾選。

項目	驗收結果	判斷
		pre-apply cleanup: Node.js upgr...
是否可進 Phase 2 change-03	暫不建議	還缺 OpenSpec status / validation output 證據。

為什麼我不建議現在直接推進 change-03 ?

因為你當初對 Antigravity 的核心要求不是「看起來改了文件」，而是**必須真的使用官方 OpenSpec 套件並通過驗證**。OpenSpec 官方 README 也明確說 OpenSpec 是在寫 code 前讓人與 AI 對齊 specs，而且每個 change 會有 proposal、specs、design、tasks；官方 Quick Start 也要求 Node.js 20.19.0+，這點本次已修正。🔗[GitHub](#)

但官方 CLI 文件中 `openspec validate` 是用來 validate changes and specs for structural issues，且支援 `--strict` 與 `--json`；因此在你批准 change-02 完成前，應該要求 Antigravity 把 `openspec status / openspec validate` 的結果回報出來，而不是只在 tasks 裡勾選前面項目。🔗[GitHub](#)

另外，ns3-gym 官方 README 的安裝流程確實使用 `ns-allinone-3.40`，並在 `ns-3.40/contrib` 下 checkout `app-ns-3.36+` branch，所以 `ns-3.40 freeze` 是合理的，這部分可以驗收。🔗[GitHub](#)

我的正式驗收判斷

Change 02 pre-apply cleanup：部分通過，尚未完全通過

可以通過的部分：

- Node.js 升級要求已被文件標記完成
- 5 個 spec validation tasks 已補上
- ns-3.40 freeze 已補上
- reproducibility tolerance 已修正
- utility provisional 已修正

尚未通過的部分：

- 0.10 `openspec status --change "ns3-baseline-benchmark" --json` 尚未完成
- 沒看到 `openspec validate ...` 的 warning / error 摘要
- 沒看到實際 `node -v`、`npm -v`、`openspec --version` 的回報內容，只看到 tasks 被勾選
- 0.11 Spec Owner 再次驗收通過 尚未完成

所以我會把狀態定為：

Change 02 artifacts 修正基本通過，但尚未 ready to enter Phase 2 change-03。請先補 OpenSpec status / validation report。

建議你回覆 Antigravity 的正式內容

請直接貼這段：

我已驗收 commit d8f093264bb51ca995df4635bd4fb02d9617e3b9。

結論：

Change 02 pre-apply cleanup 大部分已通過，但目前尚未完全完成，因此暫不批准進入 Phase 2 change-03，也不得執行 /opsx:apply。

已通過項目：

- Node.js upgrade to v20.19.0+ 已標記完成
- openspec --version 已標記確認可正常執行
- 無 EBAENGINE warning 已標記完成
- 已新增並勾選 5 個 spec validation tasks
- ns-3 target version 已修正為 ns-3.40
- 已明確禁止自行使用 ns-3.35、3.36 或 latest stable
- BBR availability check 已改為 ns-3.40 的 TcpBbr / tcp-bbr.h 檢查
- reproducibility 已改為 metric-equivalent outputs within documented tolerance
- utility score 已標明為 provisional, weights may be revised in Change 04 / Change 05 with Spec Owner approval

尚未通過項目：

- tasks.md 0.10 仍未完成：Run `openspec status --change "ns3-baseline-benchmark" --json` 後回報，確認仍為 isComplete: true
- 尚未提供 OpenSpec validation warning / error 摘要
- 尚未提供實際 node -v / npm -v / openspec --version 回報內容
- tasks.md 0.11 Spec Owner 再次驗收通過才可 /opsx:apply，目前尚未完成

請先完成以下事項，不要進入 implementation：

1. 執行並回報：

- node -v
- npm -v
- openspec --version

2. 執行並回報：

- openspec status --change "ns3-baseline-benchmark" --json
- openspec validate "ns3-baseline-benchmark" --type change --strict --json

3. 若 validation 有 warning / error，請摘要列出。

4. 若 validation 通過，請更新 tasks.md：

- 0.10 標記完成
- 0.11 保持未完成，等待 Spec Owner 最終批准

完成後請回報：

Change 02 final pre-apply validation complete, ready for Spec Owner approval.

並附上：

1. node -v
2. npm -v
3. openspec --version
4. openspec status 摘要
5. openspec validate 摘要
6. 是否有 warning / error
7. 是否需要 Spec Owner 再決策

在我再次驗收前：

- 不得執行 /opsx:apply

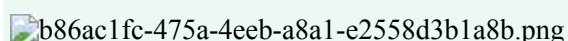
- 不得開始 ns-3 coding
- 不得執行 benchmark
- 不得建立 experiment results
- 不得寫 analysis script
- 不得進入 ns3-gym
- 不得進入 DQN / PP0
- 不得推進 Phase 2 change-03

最終回答

現在還不能推進到 Phase 2 change-03。

不是因為這次改得不好，而是因為它還缺最後一個 OpenSpec 官方驗證證據。等 Antigravity 補上 `openspec status / openspec validate` 結果，並讓 0.10 完成後，你再驗收一次；如果沒有 warning / error，我就會建議你正式批准：

Change 02 complete. Approved to proceed to Phase 2 change-03 : opengym-env proposal.

 b86ac1fc-475a-4eeb-a8a1-e2558d3b1a8b.png

<https://github.com/donuop35/ns3-drl-congestion-control/commit/ae2e535597e2088256117ff408b39791c75de14f>

Antigravity 又做完更新了並已推上 github Phase 2 change-02 的異動

請幫我驗收 git diff 是否全部 Change 02 要求都有通過

是否能推進到 Phase 2 change-03 了呢

另外 "等待 Spec Owner 最終批准 — 在批准前不得執行 /opsx:apply" 這個問題要怎麼回覆呢

/opsx:apply 這命令的意義是什麼

我可以在 Phase 2 change-03 開始前批准嗎 為什麼
顯示更多顯示較少

結論：Change 02 現在可以正式驗收通過，並可以推進到 Phase 2 change-03 : opengym-env 的 proposal / spec 階段。

但我要把兩件事分清楚：

1. 可以批准 Change 02 spec / proposal 完成。
2. 不建議現在批准 /opsx:apply。

因為 `/opsx:apply` 在 OpenSpec 官方 workflow 裡代表「開始依 tasks 進入 implementation」。官方 README 的範例流程是：`/opsx:propose` 先建立 change folder，包含 `proposal.md`、`specs/`、`design.md`、`tasks.md`；接著 `/opsx:apply` 才是「Implementing tasks...」。所以如果你現在批准 `/opsx:apply`，Antigravity 很可能就會開始 Change 02 的 ns-3 baseline coding / benchmark implementation，而不是繼續 Phase 2 的 change-03 規格設計。 [GitHub](#)

這次 commit 驗收結果

這次 commit `ae2e535...` 的主要變更很單純：把 `tasks.md` 的 `0.10` 從未完成改為完成，並明確記錄 `openspec status` 顯示 `isComplete: true`、`4/4 artifacts status: done`、6 個 spec files recognized，且 `openspec validate --strict --json` 回傳 `valid: true`，`issues: []`，`passed: 1`，`failed: 0`。

[pre-apply: mark task 0.10 compl...](#)

驗收項目	結果	判斷
Node.js v20.19.0+	通過	前一版 commit 已標記 Node.js 升級完成、 <code>openspec --version</code> 可執行、無 EBADENGINE warning。 pre-apply cleanup: Node.js upgr...
5 個 spec validation tasks	通過	baseline-methods、topology、metrics-logging、scenario-matrix、benchmark-risk-register 都已勾選驗證。 pre-apply cleanup: Node.js upgr...
ns-3.40 freeze	通過	tasks 已明確要求 ns-3.40，不得用 3.35、3.36 或 latest stable。 pre-apply cleanup: Node.js upgr...
reproducibility tolerance	通過	已改成同 seed 下 metric-equivalent outputs，不要求 byte-for-byte FlowMonitor XML。 pre-apply cleanup: Node.js upgr...
utility provisional	通過	已標示 formula provisional，權重可在 Change 04/05 經 Spec Owner approval 後調整。 pre-apply cleanup: Node.js upgr...
OpenSpec status / validate	通過	<code>isComplete: true</code> ， <code>valid: true</code> ，無 issues。 pre-apply: mark task 0.10 compl...
Task 0.11	等待你決策	仍標示 Spec Owner 再次驗收後才可 <code>/opsx:apply</code> 。 pre-apply: mark task 0.10 compl...

是否能推進到 Phase 2 change-03 ？

可以，但要用正確說法。

你可以批准：

Change 02 specification / proposal phase completed. Approved to proceed to Phase 2 change-03 : opengym-env proposal.

但不要批准：

Approved to run /opsx:apply for Change 02.

原因是你的整體流程是：

```
Phase 2：建立 OpenSpec 真相來源
change-01 project-charter
change-02 baseline-benchmark
change-03 opengym-env
change-04 drl-mvp
```

```
Phase 3：Baseline 先行
才開始真正 implementation / ns-3 baseline coding
```

所以現在應該是 **Change 02 規格完成** → **開始 Change 03 規格設計**，不是 **Change 02 規格完成** → **直接 apply 實作 baseline**。

/opsx:apply 的意義是什麼？

用白話講：

- /opsx:propose：建立或整理規格，包含 proposal / specs / design / tasks。
- /opsx:apply：照 tasks 開始做實作。
- /opsx:archive：實作完成後歸檔 change、更新 specs。

OpenSpec 官方 README 的示例就是 /opsx:propose add-dark-mode 產生 artifacts，顯示「Ready for implementation」，接著使用者輸入 /opsx:apply，AI 才開始 implementing tasks。  [GitHub](#)

所以在你的專案裡：

- 對 **Change 02** 執行 /opsx:apply = 開始做 ns-3 baseline benchmark implementation。
- 對 **Change 03** 執行 /opsx:propose = 開始建立 opengym-env 的規格文件。

你現在要的是後者。

你要怎麼回覆 AntigraVity？

請直接貼這段：

```
我已驗收 commit ae2e535597e2088256117ff408b39791c75de14f。
```

結論：

```
Change 02:ns3-baseline-benchmark 的 proposal / design / specs / tasks revisions 已通過驗收。
OpenSpec status 與 strict validation 也已通過：
- isComplete: true
- 4/4 artifacts status: done
- 6 spec files recognized
- openspec validate --strict --json → valid: true, issues: [], passed: 1, failed: 0
```

因此，Change 02 specification phase 可視為完成。

但目前不批准執行 Change 02 的 /opsx:apply。

原因：

本專案目前仍處於 Phase 2 「用 OpenSpec 建立專案真相來源」階段。

Phase 2 的目標是完成 change-01 / change-02 / change-03 / change-04 的 proposal / design / specs / tasks。

真正的 implementation / ns-3 baseline coding / benchmark execution, 應保留到 Phase 3 「Baseline 先行」再開始。

請將 tasks.md 0.11 保持為未完成或標註為：

Spec Owner approval for Change 02 specification is granted, but /opsx:apply is deferred until Phase 3.

正式批准：

Approved to proceed to Phase 2 change-03: opengym-env proposal.

不批准：

- Do not run /opsx:apply for Change 02 yet.
- Do not start ns-3 coding.
- Do not execute benchmark.
- Do not create experiment results.
- Do not write analysis scripts.
- Do not start ns3-gym implementation yet.
- Do not start DQN / PPO.

下一步請開始：

Phase 2 / OpenSpec change-03: opengym-env proposal / design / specs / tasks only.

最終狀態

Change 02：通過。

Phase 2 change-03：可以開始。

Change 02 /opsx:apply：暫不批准。

這是目前最安全的決策，因為它既承認 Change 02 規格已完成，又守住你原本定義的流程：**Phase 2 先把四個 OpenSpec change 的真相來源建立完，Phase 3 才開始 baseline implementation。**